

100 System Design & DSA Interview Questions

Simple, common-word answers for quick Full-Stack / MERN interview revision

SYSTEM DESIGN - 30 QUESTIONS

1. What is System Design?

Answer: It means planning how the parts of an application - frontend, backend, database and other services - work together.

2. What is HLD?

Answer: HLD means High-Level Design. It shows the main parts of a system and how they connect.

3. What is LLD?

Answer: LLD means Low-Level Design. It explains the small details such as classes, functions and modules.

4. What is scalability?

Answer: Scalability means the application can handle more users and more traffic without becoming too slow.

5. Horizontal vs vertical scaling?

Answer: Vertical scaling means making one server stronger. Horizontal scaling means adding more servers.

6. What is a load balancer?

Answer: It sends user requests to different servers so one server does not get overloaded.

7. What is caching?

Answer: Caching keeps frequently used data in fast memory so it can be returned quickly.

8. What is Redis?

Answer: Redis is a fast memory-based data store. It is often used for cache, sessions and queues.

9. What is database indexing?

Answer: An index helps the database find data faster.

10. What is database replication?

Answer: Replication keeps copies of the same data on more than one database server.

11. What is sharding?

Answer: Sharding splits data across multiple database servers.

12. What is a CDN?

Answer: A CDN stores files such as images and CSS closer to users so they load faster.

13. What is an API Gateway?

Answer: It is a main entry point that receives API requests and sends them to the right service.

14. What is rate limiting?

Answer: It limits how many requests a user or client can send in a certain time.

15. What is fault tolerance?

Answer: It means the system can still work even when one part fails.

16. What is availability?

Answer: Availability means how often the application is up and usable.

17. What is latency?

Answer: Latency is the time taken to get a response after sending a request.

18. What is throughput?

Answer: Throughput is how much work a system can handle in a given time.

19. Monolith vs microservices?

Answer: A monolith is one main application. Microservices split the application into smaller services.

20. How would you scale a Node.js API?

Answer: Run multiple API servers, put a load balancer in front, use cache, optimize the database and move slow work to background jobs.

21. How would you improve a slow API?

Answer: Find the slow part, optimize database queries, use indexes, reduce data, add cache and use pagination.

22. How do you design a social networking app?

Answer: Use multiple API servers, a database, cache and background workers. Add real-time messaging when needed.

23. How do you design a food-ordering app?

Answer: Keep users, restaurants, menus, carts, orders and payments as separate parts and connect them through APIs.

24. How would you prevent duplicate orders?

Answer: Use a unique request ID so the same request cannot create two orders.

25. How would you design pagination?

Answer: Show data in small pages instead of loading everything at once. For large data, cursor pagination is useful.

26. How would you handle millions of users?

Answer: Use more servers, load balancing, caching, database scaling, queues and good monitoring.

27. How would you design authentication?

Answer: Check the user's login, create a secure token or session, and protect private API routes.

28. How would you handle server failure?

Answer: Use multiple servers, health checks, load balancing and database copies so another server can continue.

29. What is a message queue?

Answer: It stores tasks or messages so another service can process them later.

30. Why use background jobs?

Answer: They handle slow tasks such as emails, notifications and reports without making the user wait.

DSA - 70 QUESTIONS

31. What is Big O?

Answer: Big O tells us how the time or memory used by an algorithm grows when the input becomes larger.

32. What is O(1)?

Answer: The work stays almost the same even when the input grows.

33. What is O(n)?

Answer: The work grows directly with the input size.

34. What is O(log n)?

Answer: The search space becomes smaller very quickly, like in binary search.

35. What is O(n²)?

Answer: The work grows roughly with the square of the input, often because of nested loops.

36. What is an array?

Answer: An array is a list of values stored in order and accessed by index.

37. How do you find the largest element?

Answer: Go through the array once and keep the largest value. Time O(n), space O(1).

38. How do you find the second largest element?

Answer: Keep track of the largest and second largest values while scanning once. Time O(n).

39. How do you remove duplicates from an array?

Answer: Use a Set to keep only unique values. Average time O(n).

40. How do you find a missing number?

Answer: Find the expected total and subtract the actual total, or use XOR. Time O(n), space O(1).

41. What is Two Sum?

Answer: Find two numbers whose sum equals a target. A Map can solve it in about O(n) time.

42. How do you move zeros to the end?

Answer: Put all non-zero values first and fill the remaining positions with zeros. Time O(n).

43. What is a palindrome?

Answer: A word or sequence that reads the same from both sides.

44. How do you check a palindrome?

Answer: Compare the first and last characters, then move toward the middle.

45. What is an anagram?

Answer: Two words are anagrams when they have the same characters with the same counts.

46. How do you check an anagram?

Answer: Count the characters in both strings or sort both strings and compare.

47. How do you find the first non-repeating character?

Answer: Count each character, then return the first character with count one.

48. What is a HashMap?

Answer: It stores data as key and value and gives very fast average lookup.

49. What is a Set?

Answer: A Set stores unique values.

50. What is binary search?

Answer: It searches a sorted array by checking the middle and removing half the search space each time.

51. What is a stack?

Answer: A stack follows LIFO - the last item added is the first item removed.

52. What is a queue?

Answer: A queue follows FIFO - the first item added is the first item removed.

53. How do you solve Valid Parentheses?

Answer: Put opening brackets in a stack and match each closing bracket with the latest opening bracket.

54. What is a linked list?

Answer: It is a list of nodes where each node points to the next node.

55. Array vs linked list?

Answer: Arrays give fast index access. Linked lists are better for some insert and delete operations.

56. How do you reverse a linked list?

Answer: Use three pointers - previous, current and next - and reverse each link.

57. How do you detect a linked-list cycle?

Answer: Use slow and fast pointers. If they meet, a cycle exists.

58. What is a binary tree?

Answer: A tree where each node can have at most two children.

59. What is a BST?

Answer: A Binary Search Tree keeps smaller values on one side and larger values on the other.

60. What are tree traversals?

Answer: Inorder is Left-Root-Right, Preorder is Root-Left-Right, Postorder is Left-Right-Root, and Level-order goes level by level.

61. BFS vs DFS?

Answer: BFS goes level by level and uses a queue. DFS goes deep first and uses recursion or a stack.

62. What is recursion?

Answer: Recursion is when a function calls itself to solve a smaller version of the same problem.

63. What is a base case?

Answer: It is the condition that stops a recursive function.

64. What is a graph?

Answer: A graph is a set of points, called vertices, connected by edges.

65. Directed vs undirected graph?

Answer: Directed edges have a direction. Undirected edges work in both directions.

66. What is a connected component?

Answer: It is a group of connected nodes where each node can reach the others.

67. What is a sliding window?

Answer: It keeps a moving range of an array or string to solve problems faster.

68. What is the two-pointer technique?

Answer: It uses two indexes that move through the data to reduce unnecessary work.

69. What is dynamic programming?

Answer: It saves answers to smaller problems so they do not have to be solved again.

70. What DSA topics are important for a junior full-stack interview?

Answer: Focus on arrays, strings, Map/Set, two pointers, sliding window, binary search, stack, queue, linked list, trees and Big O.

71. Find maximum subarray sum.

Answer: Use Kadane's algorithm and keep the best sum ending at each position. Time $O(n)$.

72. What is Kadane's algorithm?

Answer: It finds the largest sum of a continuous part of an array in $O(n)$ time.

73. How do you merge two sorted arrays?

Answer: Compare the current value in both arrays, take the smaller one, then continue.

74. What is sorting?

Answer: Sorting means arranging data in an order such as smallest to largest.

75. Bubble sort complexity?

Answer: Worst and average time are $O(n^2)$, and extra space is $O(1)$.

76. Merge sort complexity?

Answer: It takes $O(n \log n)$ time and usually $O(n)$ extra space.

77. Quick sort complexity?

Answer: Average time is $O(n \log n)$, but worst case can be $O(n^2)$.

78. What is a priority queue?

Answer: It returns the most important item first instead of simply the oldest item.

79. What is a heap?

Answer: A heap is a tree-based structure often used for priority queues.

80. What is a trie?

Answer: A trie is a tree used to store words and search prefixes quickly.

81. What is memoization?

Answer: It stores old function results so the same calculation is not repeated.

82. What is the main idea of dynamic programming?

Answer: Break the problem into smaller parts, save their answers and reuse them.

83. What is a greedy algorithm?

Answer: It chooses the best-looking option at each step.

84. When can you use binary search?

Answer: When the data or search condition is ordered so half the search space can be removed each time.

85. What is a time-space trade-off?

Answer: It means using more memory to make the program faster, or using less memory and more time.

86. What is recursion stack space?

Answer: Every recursive call uses memory on the call stack. Too much recursion can use a lot of memory.

87. How do you find the middle of a linked list?

Answer: Use slow and fast pointers. Slow moves one step and fast moves two steps.

88. How do you find the depth of a binary tree?

Answer: Find the depth of left and right sides and take the larger one, plus one.

89. What is BFS time complexity?

Answer: For an adjacency-list graph, BFS is $O(V + E)$.

90. What is DFS time complexity?

Answer: For an adjacency-list graph, DFS is $O(V + E)$.

91. What is a priority queue?

Answer: It removes the highest- or lowest-priority item first. A heap can implement it.

92. What is a heap?

Answer: It keeps the smallest or largest value near the top, depending on the heap type.

93. What is a trie?

Answer: It stores words by their letters and makes prefix search fast.

94. What is memoization?

Answer: It saves already calculated results and reuses them.

95. What is a greedy algorithm?

Answer: It makes the best choice at the current step instead of checking every future choice.

96. What is a time-space trade-off?

Answer: You may use extra memory to make a program faster, or save memory by doing more work.

97. How do you find the middle of a linked list?

Answer: Use a slow pointer and a fast pointer; when fast reaches the end, slow is near the middle.

98. How do you find the depth of a binary tree?

Answer: Use recursion or BFS and count the number of levels.

99. What is the time complexity of BFS?

Answer: With an adjacency list, it is $O(V + E)$.

100. What is the time complexity of DFS?

Answer: With an adjacency list, it is $O(V + E)$.